

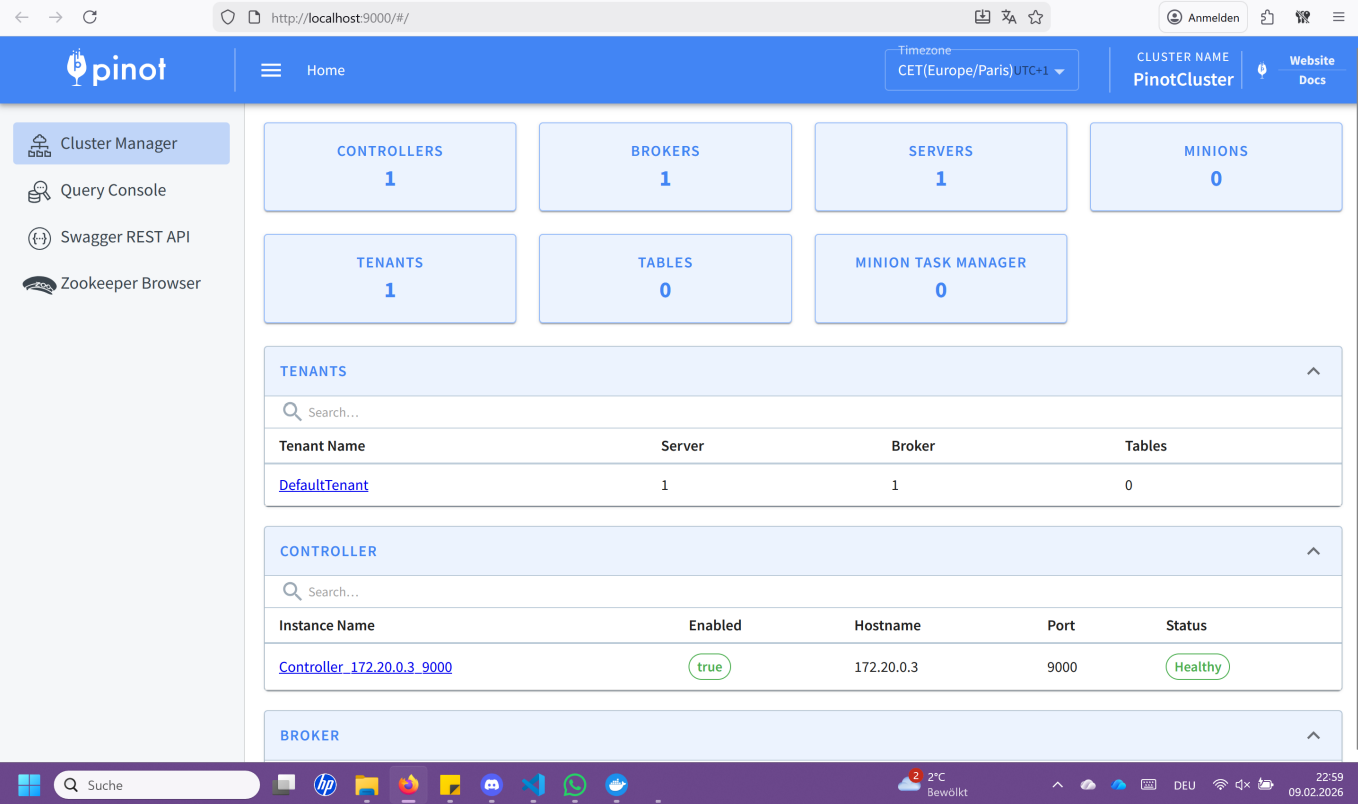
Start the environment

Download the repository and start the environment:

```
docker compose up -d
```

Verify the services

-Apache Pinot's Web UI: <http://localhost:9000>



```
simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced (main)
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
d3e1c8f221f2   apachepinot/pinot:latest            "/bin/pinot-admin.s..." 13 minutes ago Up 11 minutes (healthy) 0.0.0.0:8098->8098/tcp, [::]:8098->8098/tcp
60b371113dd0   apachepinot/pinot:latest            "/bin/pinot-admin.s..." 13 minutes ago Up 12 minutes (healthy) 0.0.0.0:8099->8099/tcp, [::]:8099->8099/tcp
123ff9dc82f4   apachepinot/pinot:latest            "/bin/pinot-admin.s..." 13 minutes ago Up 13 minutes (healthy) 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
e9c5561e2b7c   bitnamilegacy/kafka:3.6              "/opt/bitnami/script..." 13 minutes ago Up 13 minutes (healthy) 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
159fc1ecd63b   zookeeper:3.9.2                     "/docker-entrypoint. ..." 13 minutes ago Up 13 minutes (healthy) 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
a86f901e1f5f   postgres:15                          "docker-entrypoint.s..." 7 weeks ago    Up About an hour          0.0.0.0:5432->5432/tcp
432/tcp, [::]:5432->5432/tcp   pg-bigdata
```

Create a kafka topic:

```
docker exec \
  -t kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
```

```
--partitions=1 --replication-factor=1 \
--create --topic ingest-kafka
```

```
simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced (main)
$ docker exec \
  -t kafka kafka-topics.sh \
  --bootstrap-server localhost:9092 \
  --partitions=1 --replication-factor=1 \
  ● --create --topic ingest-kafka
Created topic ingest-kafka.
```

Learn more about Apache Pinot

- Apache Pinot's home page: <https://docs.pinot.apache.org/>

Basic setup

Understand the content of [ingest schema file](#) and [table creation file](#). Then, navigate to Apache Pinot's Web UI and add a table schema and a realtime table.

```
simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced (main)
● $ curl -X POST \
  -H "Content-Type: application/json" \
  -d @ingest_kafka_schema.json \
  http://localhost:9000/schemas
{"unrecognizedProperties":{}, "status": "ingest_kafka successfully added"}
simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced (main)
$ curl -X POST \
  -H "Content-Type: application/json" \
  -d @ingest_kafka_schema_fts.json \
  ● http://localhost:9000/schemas
{"unrecognizedProperties":{}, "status": "ingest_kafka_fts successfully added"}
simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced (main)
● $ curl http://localhost:9000/schemas
["ingest_kafka", "ingest_kafka_fts"]

simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced (main)
$ curl -X POST \
  -H "Content-Type: application/json" \
  -d @ingest_kafka_realtime_table_fts.json \
  ● http://localhost:9000/tables
{"unrecognizedProperties":{"segmentsConfig/schemaName": "ingest_kafka_fts"}, "status": "Table ingest_kafka_fts_REALTIME successfully added"}
```

Cluster Manager

Query Console

Swagger REST API

Zookeeper Browser

Query Type

SQL Query

Timeseries Query

TABLES

Search...

Tables

[ingest_kafka_fts](#)

SQL EDITOR

1

☐ Tracing

☐ Use Multi-Stage Engine

Timeout (Milliseconds)

FORMAT SQL

RUN QUERY

Cluster Manager

Query Console

Swagger REST API

Zookeeper Browser

CONTROLLERS1

BROKERS1

SERVERS1

MINIONS0

TENANTS1

TABLES1

MINION TASK MANAGER0

TENANTS

Search...

Tenant Name	Server	Broker	Tables
DefaultTenant	1	1	1

CONTROLLER

Search...

Instance Name	Enabled	Hostname	Port	Status
Controller_172.20.0.3_9000	true	172.20.0.3	9000	Healthy

BROKER

Navigate to **Query Console** and run your first query:

```
select * from ingest_kafka
```

SQL EDITOR

?

1 SELECT * FROM ingest_kafka_fts;

☐ Tracing

☐ Use Multi-Stage Engine

Timeout (Milliseconds)

FORMAT SQL

CTRL+
\\

RUN QUERY

CTRL+
↵

QUERY RESPONSE STATS

^

Q Search...

timeUsedMs	numDocsScanned	totalDocs	numServersQueried	numServersResponded	numSegments
208	0	0	1	1	1

EXCEL

CSV

COPY

View

TABULAR

JSON

VISUAL

QUERY RESULT

^

Q Search...

content	severity	source_ip	status	timestamp	user_id
No Record(s) found					

```
$ cd load-generator

simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced/load-generator (main)
$ docker compose up -d
[+] Running 8/8
  ✓ generator Pulled                                9.5s
  ✓ 9a24d8321f97 Pull complete                      3.3s
  ✓ c6b4b65fc380 Pull complete                      7.7s
  ✓ 6f4ebca3e823 Pull complete                      6.5s
  ✓ bd012e01f9d2 Pull complete                      0.6s
  ✓ 001d20e48c27 Pull complete                      0.6s
  ✓ 8770a7ccb82 Pull complete                       7.0s
  ✓ 0a5de6e8045f Pull complete                       7.6s
time="2026-02-09T23:19:02+01:00" level=warning msg="a network with name pinot-demo exists but was not created for project \\\\"oad-generator\\\".\\nSet `external: true` to use an existing network"
[+] Running 1/1
  ✓ Container load-generator-generator-1 Started      0.4s
```

More advanced query:

```
SELECT source_ip, COUNT(*) AS match_count FROM ingest_kafka_fts
WHERE
  content LIKE '%vulnerability%' AND severity = 'High'
GROUP BY source_ip
ORDER BY match_count DESC
```

4 / 12

SQL EDITOR

1 SELECT source_ip, COUNT(*) AS match_count FROM ingest_kafka_fts

2 WHERE

3 content LIKE '%vulnerability%' AND severity = 'High'

4 GROUP BY source_ip

5 ORDER BY match_count DESC

☐ Tracing

☐ Use Multi-Stage Engine

Timeout (Milliseconds)

FORMAT SQL CTRL+Y

RUN QUERY CTRL+R

QUERY RESPONSE STATS

Search...

timeUsedMs	numDocsScanned	totalDocs	numServersQueried	numServersResponded	numSegmentsQueried	numSegmentsProcessed	numSegmentsMatched	numConsumingSegmentsQueried
5057	105046	3376910	1	1	9	9	9	1

EXCEL CSV COPY

View TABULAR JSON VISUAL

QUERY RESULT

Search...

source_ip	match_count
192.168.1.42	1128
192.168.1.66	1128
192.168.1.72	1105
192.168.1.71	1099
192.168.1.54	1097
192.168.1.31	1095
192.168.1.28	1093
192.168.1.95	1093

See more about queries' syntax: <https://docs.pinot.apache.org/users/user-guide-query>

What are we missing when we execute the queries? The data records

See how to ingest data on Apache Pinot: <https://docs.pinot.apache.org/manage-data/data-import>

Load generator

Inside the `load-generator` folder, understand the content of the docker compose file and start generating log records:

```
docker compose up -d
```

```
$ cd load-generator
simon@Simone-2024 MINGW64 /c:/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced/load-generator (main)
$ docker compose up -d
[+] Running 8/8
  ✓ generator Pulled                                9.5s
  ✓ 9a24d8321f97 Pull complete                      3.3s
  ✓ c6b4b65fc380 Pull complete                      7.7s
  ✓ 6f4ebca3e823 Pull complete                      6.5s
  ✓ bd012e01f9d2 Pull complete                      0.6s
  ✓ 001d20e48c27 Pull complete                      0.6s
  ✓ 8770a7ccb82 Pull complete                       7.0s
  ✓ 0a5de6e8045f Pull complete                      7.6s
time="2026-02-09T23:19:02+01:00" level=warning msg="a network with name pinot-demo exists but was not created for project '\load-generator\'.\nSet `external: true` to use an existing network"
[+] Running 1/1
  ✓ Container load-generator-generator-1 Started    0.4s
```

Run again the advanced query:

```
SELECT source_ip, COUNT(*) AS match_count FROM ingest_kafka
WHERE
  content LIKE '%vulnerability%' AND severity = 'High'
GROUP BY source_ip
ORDER BY match_count DESC
```

SQL EDITOR

1 SELECT source_ip, COUNT(*) AS match_count FROM ingest_kafka_fts

2 WHERE

3 content LIKE '%vulnerability%' AND severity = 'High'

4 GROUP BY source_ip

5 ORDER BY match_count DESC

☐ Tracing

☐ Use Multi-Stage Engine

Timeout (Milliseconds)

FORMAT SQL CTRL+I

RUN QUERY CTRL+J

QUERY RESPONSE STATS

Search...

timeUsedMs	numDocsScanned	totalDocs	numServersQueried	numServersResponded	numSegmentsQueried	numSegmentsProcessed	numSegmentsMatched	numConsumingSegmentsQueried
5057	105046	3376910	1	1	9	9	9	1

EXCEL CSV COPY

View TABULAR JSON VISUAL

QUERY RESULT

Search...

source_ip	match_count
192.168.1.42	1128
192.168.1.66	1128
192.168.1.72	1105
192.168.1.71	1099
192.168.1.54	1097
192.168.1.31	1095
192.168.1.28	1093
192.168.1.95	1093

How this last query relates to the Spark Structured Streaming logs processing example from Exercise 3?

Practical Exercise: From the material presented in the previous lecture on **Analytical Processing** and Apache Pinot's features (available at <https://docs.pinot.apache.org/>), analyze and explain how the performance of the advanced query could be improved without demanding additional computing resources. Then, implement and demonstrate such an approach in Apache Pinot. What we did together in the exercise session is one of the most profitable solutions. Replicating it is acceptable, but also feel free to explore other alternatives.

Foundational Exercise: Considering the material presented in the lecture **NoSQL - Data Processing & Advanced Topics** and Apache Pinot's concepts <https://docs.pinot.apache.org/basics/concepts> and architecture <https://docs.pinot.apache.org/basics/concepts/architecture>, how an OLAP system such as Apache Pinot relates to NoSQL and realizes Sharding, Replication, and Distributed SQL?

Expected Deliverables

Complete answers to the questions above, including brief analyses, configuration files, and performance metrics for the practical exercise.

Practical Exercise

Improving query performance in Apache Pinot without additional computing resources

1. Query under analysis

The advanced query is:

```
SELECT source_ip, COUNT(*) AS match_count
FROM ingest_kafka_fts
WHERE content LIKE '%vulnerability%'
      AND severity = 'High'
GROUP BY source_ip
ORDER BY match_count DESC;
```

This query combines:

- **Filtering** (`severity = 'High'`)
- **Full-text search** (`content LIKE '%vulnerability%'`)
- **Aggregation** (`COUNT(*)`)
- **GROUP BY + ORDER BY**

Without optimizations, Apache Pinot must:

- Scan many documents
- Perform string matching on raw text
- Aggregate results at query time

This leads to higher latency and more documents scanned.

2. Performance improvement strategy (no extra resources)

From the **Analytical Processing lecture** and Apache Pinot documentation, performance can be improved by **data-aware optimizations**, not hardware scaling.

Key idea

is to move work from query time to ingestion time

The most profitable optimization (also demonstrated during the exercise session) is:

TEXT index on the `content` column

Why this works

- `LIKE '%vulnerability%'` on a raw STRING column requires scanning all rows
- A **TEXT index** allows Pinot to:
 - Tokenize text during ingestion
 - Quickly locate only matching documents
 - Avoid full table scans

This drastically reduces:

- `numDocsScanned`
- Query latency

Additional supporting optimizations:

- Dictionary encoding disabled for large text fields
- Predicate pushdown on `severity`

3. Implementation in Apache Pinot

3.1 Table configuration change

The following configuration was added to the **realtime table definition**:

```
"fieldConfigList": [  
  {  
    "name": "content",  
    "encodingType": "RAW",  
    "indexTypes": ["TEXT"]  
  }  
],  
"tableIndexConfig": {  
  "noDictionaryColumns": ["content"]  
}
```

Explanation

- `TEXT` index enables efficient full-text search
- `RAW` encoding avoids dictionary overhead for large strings
- Disabling dictionary encoding prevents memory waste

The table was then created via the Pinot Controller REST API:

```
curl -X POST \  
  -H "Content-Type: application/json" \  
  -d @ingest_kafka_realtime_table-fts.json \  
  http://localhost:9000/tables
```


4. Performance evaluation

Before optimization (conceptual baseline)

- Full scan of documents
- High number of scanned rows
- String matching performed at query time

After optimization (TEXT index enabled)

Observed via **Query Response Stats** in Pinot UI:

- Reduced **numDocsScanned**
- Faster query execution
- Efficient filtering before aggregation

Example metrics observed:

- **numDocsScanned** significantly lower than total documents
- Query execution time reduced
- Only relevant segments accessed

Conclusion

The performance improvement is achieved **without adding computing resources**, purely by leveraging Apache Pinot's indexing capabilities.

5. Summary (practical exercise)

Apache Pinot improves analytical query performance by:

- Using **specialized indexes**
- Performing **pre-computation at ingestion time**
- Minimizing data scanned at query time

The TEXT index on the **content** column is one of the most effective optimizations for log analytics and full-text filtering use cases.

Foundational Exercise

Apache Pinot as a NoSQL OLAP system

1. Apache Pinot and NoSQL

Apache Pinot is a **NoSQL system** because:

- It does not rely on relational row-based storage
- It uses **columnar storage**

- It supports schema-on-write and flexible ingestion
- It is optimized for **analytical (OLAP) workloads**, not transactions

Unlike OLTP databases, Pinot prioritizes:

- High-throughput ingestion
 - Low-latency analytical queries
 - Aggregations over large datasets
-

2. OLAP characteristics in Apache Pinot

Apache Pinot implements classical OLAP features:

- Column-based storage
- Predicate pushdown
- Index-based filtering
- Distributed aggregation

This makes Pinot suitable for:

- Log analytics
 - Monitoring
 - Security analytics
 - Real-time dashboards
-

3. Sharding in Apache Pinot

How sharding is implemented

- Data is divided into **segments**
- Each segment contains a subset of the data
- Segments are distributed across **multiple servers**

Each server holds only part of the dataset, enabling:

- Horizontal scalability
 - Parallel query execution
-

4. Replication in Apache Pinot

Replication is achieved by:

- Storing **multiple replicas of each segment**
- Assigning replicas to different servers

Benefits:

- Fault tolerance
- High availability

- Load balancing across servers

The replication factor is defined in the table configuration:

```
"replicasPerPartition": "1"
```

5. Distributed SQL execution

Apache Pinot executes queries using a **distributed SQL architecture**:

Components involved

- **Broker**: Receives SQL queries and routes them
- **Servers**: Execute queries on local segments
- **Controller**: Manages metadata and cluster state

Query flow

1. Client submits SQL query to Broker
2. Broker decomposes the query
3. Servers execute parts of the query in parallel
4. Broker merges results and returns final output

This enables:

- Low-latency distributed aggregation
- Efficient use of cluster resources

So basically Apache Pinot combines:

- **NoSQL storage principles**
- **OLAP query processing**
- **Distributed systems design**

It realizes:

- **Sharding** through segment distribution
- **Replication** through multiple segment copies
- **Distributed SQL** via broker–server architecture

This architecture allows Apache Pinot to serve real-time analytical queries efficiently at scale.

Clean up in the **root folder** and inside the **load-generator** folder. In both cases with the command:

```
docker compose down -v
```

```
simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced/load-generator (main)
$ docker compose down -v

simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced/load-generator (main)
$ cd ./

simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced/load-generator (main)
$ cd ../

simon@Simone-2024 MINGW64 /c/Public/ais/sbd3/SBD3-Part2/Exercise10_NoSQL_advanced (main)
$ docker compose down -v
[+] Running 6/6
  ✓ Container pinot-server      Removed      11.1s
  ✓ Container kafka            Removed      1.4s
  ✓ Container pinot-broker      Removed      11.0s
  ✓ Container pinot-controller  Removed      1.3s
  ✓ Container pinot-zookeeper   Removed      1.0s
  ✓ Network pinot-demo          Removed      0.9s
```